

FINAL YEAR PROJECT REPORT

Project title:

**Design and development of a multi-agent system for queue
waiting-time estimation from video streams**

Prepared by:

Salmen Ben Ammar

Host Company:



Company Supervisor:

Ramzi Guesmi

Academic Supervisor:

Tarek Hamrouni

Academic Year 2025–2026



Dedications

To my parents,

No dedication could ever truly reflect my unending love and gratitude for all you've done for my education and well-being. May you recognize in this simple work the fulfillment of your dreams and the reward of your many sacrifices.

To my sister,

Your affection, support, and constant encouragement have been an immense source of motivation throughout my journey. Your belief in me helped me overcome challenges and stay focused on my goals. I wish you a life filled with success and happiness.

To my family,

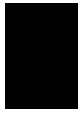
Your presence and unconditional love are my greatest treasures. Thank you for everything. You will forever remain my inexhaustible source of support, love, and comfort.

To all my friends,

My second family who made my journey an indescribable experience, and who have always encouraged me and believed in me. I wish you a future filled with achievement and joy.

To all of you

I dedicate this work.



Acknowledgements

At the conclusion of this project, I wish to express my sincere gratitude to the individuals who have greatly contributed to its successful completion.

First, I would like to express my profound appreciation to the entire team at **Data-DoIt** for graciously providing me the opportunity to carry out this final-year project.

I would also like to extend my sincere thanks to my professional supervisor, **Mr. Ramzi Guesmi**, for his unwavering support, constant availability, patience, and his relevant and constructive guidance, which proved invaluable throughout this project.

I warmly thank my academic supervisor, **Mr. Tarek Hamrouni**, for his constant availability, patience, and insightful advice, which were essential for the successful completion of this work. His guidance, encouragement, and constructive feedback not only helped me overcome numerous challenges but also inspired me to strive for excellence and deepen my understanding throughout this project.

I would also like to take this opportunity to express my gratitude to the entire faculty of **Sesame University**, whose contributions have been fundamental to my education over the past three years.

Finally, I sincerely thank the members of the jury for agreeing to evaluate my work. Their expertise and commitment are greatly appreciated, and I wish to express my deep gratitude for their contribution to the assessment of this project.



Table of Content

List of Figures	ix
List of Tables	x
General Introduction	xi
1 Context of the Project	1
1.1 Introduction	2
1.2 Host Organization	2
1.2.1 DataDoIt	2
1.2.2 Sectors of Activity	3
1.3 Presentation of the Project	4
1.3.1 Project Context	4
1.3.2 Problem Statement	4
1.3.3 Study of Existing Approaches	5
1.3.4 Proposed Solution	6
1.3.5 Objectives of the Project	7
1.3.6 Project Scope and Boundaries	7
1.3.7 Target Audience	8
1.4 Project Management Methodology	9
1.4.1 Choice of Methodology: Agile	9
1.4.2 SCRUM	9
1.4.2.1 Module A: Backend Core (Sprints 1-3)	9
1.4.2.2 Module B: Web Experience (Sprints 4-5)	9

1.4.2.3	Module C: Automation and Finalization (Sprints 6-8) . .	10
1.5	Conclusion	10
2	Requirements Specification and Analysis	12
2.1	Introduction	13
2.2	Requirements Specification	13
2.2.1	Identification of Actors	13
2.2.2	Functional Requirements	13
2.2.3	Non-functional Requirements	14
2.3	Product Backlog	15
2.4	Global Use Case Diagram	17
2.5	Sprint Scope Overview	17
2.6	Data Modeling	18
2.6.1	Core Entities	18
2.6.2	Data Lifecycle and Persistence Strategy	18
2.7	Project Environment	20
2.7.1	Hardware and Software Environment	20
2.7.2	Technologies Used	21
2.8	Software Architecture	22
2.8.1	Overall Architecture	22
2.8.2	Backend Architecture	23
2.8.3	Frontend Architecture	23
2.9	Conclusion	23
3	Sprint 1: Foundation and Perception	24
3.1	Introduction	25
3.2	Sprint Backlog	25
3.3	Sprint Analysis	26

3.3.1	Detailed Use Case Diagram	26
3.3.2	Use Case Scenarios	26
3.4	Design	27
3.4.1	Activity Diagram	27
3.4.2	Sequence Diagrams	27
3.5	Implementation	28
3.5.1	Detection Layer	28
3.5.2	Tracking Layer	28
3.5.3	Zone-Aware Filtering	28
3.5.4	Main Loop Integration	28
3.6	Conclusion	29
4	Sprint 2: Analytics and Uncertainty	30
4.1	Introduction	31
4.2	Sprint Backlog	31
4.3	Design	31
4.3.1	Component Diagram	31
4.3.2	Data Flow Diagram	32
4.3.3	Sequence Diagrams	32
4.4	Implementation	33
4.4.1	Queue Event Windows and Rate Estimation	33
4.4.2	Wait-Time Logic and Stability Handling	33
4.4.3	Runtime Contract for Monitoring and Automation	34
4.5	Conclusion	34
5	Sprint 3: Advanced Connectivity and API	35
5.1	Introduction	36
5.2	Sprint Backlog	36

5.3	Design	37
5.3.1	Component Diagram	37
5.3.2	Data Flow Diagram	37
5.3.3	Sequence Diagrams	38
5.4	Implementation	38
5.4.1	RTSP Source Integration	38
5.4.2	ONVIF Discovery and Stream Resolution	39
5.4.3	FastAPI BFF and Typed Contract	39
5.4.4	Transport Capability and WebRTC Signaling	39
5.4.5	Runtime Orchestration and Real-Time Events	40
5.4.6	Persistence and Recovery	41
5.5	Conclusion	41
6	Sprint 4: Web Design and Scaffolding	42
6.1	Introduction	43
6.2	Sprint Backlog	43
6.3	Design	43
6.3.1	UI and UX Design	43
6.3.2	Frontend Architecture	44
6.3.3	Design Rationale	44
6.4	Implementation	45
6.5	Conclusion	45
7	Sprint 5: Frontend Integration and Live Monitoring	46
7.1	Introduction	47
7.2	Sprint Backlog	47
7.3	Design	48
7.3.1	Data Flow Diagram	48

7.3.2	Sequence Diagrams	48
7.4	Implementation	49
7.4.1	Typed API Contract Integration	49
7.4.2	Dashboard State Orchestration	49
7.4.3	WebSocket Reliability and Fallback Logic	49
7.4.4	Transport Capability Routing and WebRTC Lifecycle	50
7.4.5	Dashboard UI Integration	51
7.4.6	Integration Testing	51
7.5	Conclusion	51
8	Sprint 6: n8n Automation and Alerting	52
8.1	Introduction	53
8.2	Sprint Backlog	53
8.3	Design	54
8.3.1	Component Diagram	54
8.3.2	Data Flow Diagram	54
8.4	Implementation	55
8.4.1	Webhook Payload Contract	55
8.4.2	Reliable Webhook Delivery	55
8.4.3	Runtime Alert Dispatch Integration	55
8.4.4	n8n Workflow Routing and Telegram Delivery	56
8.4.5	Automation Testing Artifacts	56
8.5	Conclusion	56
9	Sprint 7: Evaluation and Optimization	57
9.1	Introduction	58
9.2	Sprint Backlog	58
9.3	Design	58

9.3.1	Test Plan	58
9.3.2	Performance Metrics	59
9.4	Implementation	59
9.4.1	Automated Test Coverage	59
9.4.2	Runtime Optimization Actions	59
9.4.3	Resilience Improvements	60
9.4.4	Observed Outcome	60
9.5	Conclusion	60
10	General Conclusion	61
10.1	Summary	62
10.2	Objective Review	62
10.3	Future Work	63
A	Appendices	65
A.1	Useful Commands	65
A.2	Example Webhook Payload	65
A.3	Items to Add Before Final Submission	66



List of Figures

2.1	Global use case diagram of the queue wait-time estimation system. . . .	17
2.2	Unified SQLite schema used for feed metadata, queue sessions, and alert archival.	19
3.1	Sprint 1 use case diagram for detection and tracking in queue zones. . .	26
3.2	Sprint 1 activity flow from frame ingestion to queue-zone updates. . . .	27
3.3	Sprint 1 sequence diagram from video ingestion to queue-state updates.	27
4.1	Sprint 2 components for queue analysis and stability handling.	31
4.2	Data flow from tracked events to queue metrics and stability state. . . .	32
4.3	Sprint 2 sequence diagram for queue rate estimation and stability handling.	32
5.1	Sprint 3 components for camera integration and backend services.	37
5.2	Data flow from video feeds to API responses and WebSocket broadcasts.	37
5.3	Sprint 3 sequence diagram for feed orchestration and live event distribution.	38
6.1	Frontend architecture for routing, components, and state boundaries. . .	44
7.1	Data flow between backend services and frontend monitoring components.	48
7.2	Sprint 5 sequence diagram for dashboard updates and alert surfacing. .	48
8.1	Components involved in backend-to-n8n-to-Telegram automation flow.	54
8.2	Alert payload flow from backend metrics to external notifications. . . .	54



List of Tables

1.1	Collaboration perimeter with DataDoIt	3
1.2	From operational pain points to system-level requirements	5
1.3	Positioning of the proposed solution against common approach families	6
1.4	Objective-to-module traceability	8
1.5	Scrum module mapping used in this report	10
2.1	Actor responsibility matrix	14
2.2	Functional requirement traceability matrix	15
2.3	Non-functional requirement quality matrix	16
2.4	Backlog domain structuring	17
2.5	Primary global use cases	18
2.6	Project sprint scope overview	19
2.7	Conceptual entity mapping	20
2.8	Execution environment profile	21
2.9	Architecture layer-to-module mapping	22



General Introduction

In a context where digital transformation and artificial intelligence increasingly shape operational decision-making, service organizations are under strong pressure to reduce waiting time and improve service quality. Supermarkets, banks, and service centers face the same recurring challenge: queues are dynamic, difficult to monitor manually, and highly sensitive to demand fluctuations.

In practice, customers often join a queue without a clear idea of expected waiting time, while managers and operational staff do not always have reliable real-time indicators to adapt resources quickly. Manual observation remains subjective and costly, while simple counting solutions do not provide robust waiting-time estimation.

This end-of-studies project addresses this gap through the design and development of an intelligent real-time queue monitoring system based on computer vision and queueing theory. The proposed system detects and tracks people from video streams, analyzes occupancy inside configurable queue zones, estimates arrival and service rates, computes waiting time, and exposes actionable metrics for operational use.

This work combines classical queueing theory with modern computer-vision detection and tracking techniques [1], [2] to provide actionable waiting-time estimates.

This report is organized as follows. Chapter 1 presents the context of the project, host organization, problem statement, and objectives. Chapter 2 defines requirements specification and analysis, including architecture choices. Chapters 3 to 9 present sprint-by-sprint implementation progress from perception foundations to evaluation and optimization. Chapter 10 provides the general conclusion and future work directions.

Context of the Project

Table of Content

1.1	Introduction	2
1.2	Host Organization	2
1.2.1	DataDoIt	2
1.2.2	Sectors of Activity	3
1.3	Presentation of the Project	4
1.3.1	Project Context	4
1.3.2	Problem Statement	4
1.3.3	Study of Existing Approaches	5
1.3.4	Proposed Solution	6
1.3.5	Objectives of the Project	7
1.3.6	Project Scope and Boundaries	7
1.3.7	Target Audience	8
1.4	Project Management Methodology	9
1.4.1	Choice of Methodology: Agile	9
1.4.2	SCRUM	9
1.5	Conclusion	10

1.1 Introduction

This chapter positions the Queue Wait-Time Estimation System in its operational, organizational, and methodological context. Beyond a general introduction, it clarifies why queue monitoring remains a high-impact problem for service organizations and why an end-to-end technical solution is required instead of isolated counting tools.

The chapter also establishes the project baseline used in the rest of the report: collaboration context with DataDoIt, problem formalization, comparison with existing approaches, solution scope, and implementation governance through Scrum modules. This framing is intentionally detailed because later technical chapters depend on these assumptions.

1.2 Host Organization

1.2.1 DataDoIt

This end-of-studies project was carried out in collaboration with DataDoIt, a data-driven sustainability consulting firm headquartered at Elghazal Technopark in Tunis, Tunisia. DataDoIt specializes in transforming environmental and sustainability commitments into measurable, actionable outcomes for governments, municipalities, and enterprises across the EMEA region. The company operates at the intersection of data science and sustainability, offering services that span decarbonization strategy, ESG reporting, smart resource analytics, and circular economy implementation. Its approach, summarized as Measure It, Optimize It, Do It, reflects a methodology grounded in quantitative assessment, strategic planning, and hands-on technical deployment. To date, the company has delivered over 15 active projects across 6 key sectors, with a reported average ROI of 2.3x and a 94% client retention rate.

The collaboration provided a concrete engineering context in which data collection,

real-time monitoring, and automated analytics are central concerns, requirements that directly shaped the design and scope of this project.

Table 1.1: Collaboration perimeter with DataDoIt

Dimension	Scope in this project
Business context	Queue management in service scenarios where waiting time and staffing decisions are operationally sensitive.
Engineering constraints	Multi-source video ingestion, variable visual conditions, low-latency monitoring, and integration with automation channels.
Expected outputs	Actionable queue indicators, wait-time estimation, and alert-ready payloads for external workflows.
Validation mode	Iterative sprint reviews using implementation evidence from backend modules, API contracts, frontend behavior, and automation tests.

1.2.2 Sectors of Activity

DataDoIt delivers sustainability solutions across six mission-critical sectors where data-driven environmental performance has direct operational and financial implications:

- government and public sector institutions seeking to translate climate policy into measurable action,
- smart cities and municipalities requiring integrated urban resilience and climate adaptation systems,
- industrial and manufacturing operations targeting Scope 1-3 carbon footprint reduction and supply chain efficiency,
- real estate and infrastructure organizations pursuing green certification and climate-resilient asset development,
- financial services firms navigating ESG compliance and green finance opportunities,

- energy and utilities providers accelerating the transition toward renewable integration and grid modernization.

1.3 Presentation of the Project

1.3.1 Project Context

Manual queue supervision is often subjective, difficult to scale, and highly dependent on continuous manual attention. In many environments, monitoring remains reactive: teams observe congestion after it appears rather than anticipating it early with measurable indicators.

This project addresses that gap by transforming camera streams into decision-oriented telemetry. The target output is not only occupancy counts, but a coherent set of operational metrics: people in zone, arrival rate, service rate, expected wait time, queue stability state, and alert states consumable by monitoring and automation layers.

1.3.2 Problem Statement

Existing operational practices and basic counting tools present recurring limitations:

- they provide limited anticipation of congestion,
- they rarely estimate waiting time explicitly,
- they provide weak visibility into queue stability during sudden demand spikes,
- they are often disconnected from automated response workflows.

As a consequence, interventions (staff reallocation, lane opening, escalation) can be delayed or triggered on incomplete signals. The problem is therefore both analytical and integrational: producing robust indicators and delivering them through actionable channels.

Table 1.2: From operational pain points to system-level requirements

Observed pain point	Operational impact	Required system capability
Manual visual estimation is inconsistent across shifts.	Queue state interpretation varies between staff members, reducing decision reliability.	Standardized per-feed metrics with explicit formulas and stable update cadence.
Occupancy-only monitoring is descriptive but not predictive.	Teams react late because no forward-looking wait-time signal is available.	Continuous estimation of λ , μ , and expected wait time W .
Stability state is not explicit during bursty periods.	Teams may overreact or underreact when service and arrivals fluctuate quickly.	Stable/unstable queue-state classification with fallback wait-time logic.
Alerting tools are not integrated with runtime analytics.	Escalation depends on manual relay, increasing response latency.	Automated webhook dispatch and workflow-based notification routing.

1.3.3 Study of Existing Approaches

Recent computer-vision frameworks provide strong building blocks for person perception, notably YOLO-based detection and tracking algorithms such as ByteTrack [2], [3]. Queueing theory, in parallel, offers formal models for service-system behavior, including interpretable relationships between arrival rate, service rate, and expected waiting time [1].

However, many practical deployments remain partial. Some focus on counting without identity continuity, others on visualization without explicit stability semantics, and others on analytics without operational delivery channels. In addition, integration boundaries between perception pipelines, web supervision, and alert automation are frequently treated as separate projects instead of one coherent architecture.

Table 1.3: Positioning of the proposed solution against common approach families

Approach family	Strengths	Typical limitation
Manual or occupancy-only monitoring	Simple to deploy and easy to interpret at first glance.	Weak predictive capability and high human subjectivity.
Vision detection without persistent tracking	Fast frame-level counting.	Identity discontinuity can distort arrival and departure logic.
Analytical queue models without vision coupling	Strong mathematical interpretability.	Requires reliable event extraction pipeline from real scenes.
Proposed integrated pipeline	Couples perception, queue analytics, web monitoring, and automation.	Requires careful multi-layer integration and validation.

1.3.4 Proposed Solution

The proposed system combines perception, analytics, and integration in one modular pipeline:

1. person detection and multi-object tracking from camera feeds,
2. polygon-based queue-zone filtering,
3. estimation of λ , μ , and expected wait time,
4. stable/unstable queue-state interpretation for safe operational decisions,

5. real-time web supervision through tiered live transport (WebRTC primary, MJPEG fallback),
6. automation through n8n webhooks and Telegram alerts.

This architecture aims to convert visual observations into operational actions while keeping each layer independently testable. Detection/tracking modules can evolve without rewriting queue formulas; web and automation channels can evolve without changing the core metric contract.

1.3.5 Objectives of the Project

The project objectives are to:

1. detect and track people in real time,
2. count people in configurable queue zones,
3. estimate arrival and service rates,
4. compute expected wait time,
5. classify queue stability for operational reliability,
6. expose metrics through web interfaces and automation workflows.

To keep implementation traceable, objectives are mapped to delivery modules and observable outputs.

1.3.6 Project Scope and Boundaries

The implementation scope includes backend perception and analytics modules, API/runtime integration, web monitoring, and external workflow automation. The report deliberately focuses on this web-first operational architecture.

All implementation details and validation narratives are presented through the

Table 1.4: Objective-to-module traceability

Objective ID	Objective	Primary module	Observable output
OBJ-01	Real-time detection and tracking	Module A	Stable in-frame detections with persistent tracker IDs.
OBJ-02	Queue-zone counting	Module A	Per-frame in-zone people count and zone-aware overlays.
OBJ-03	Queue metric estimation (λ, μ, W)	Module A	Continuous metric payloads from queue analysis pipeline.
OBJ-04	Queue stability handling	Module A	Stable/unstable state classification and robust fallback wait-time behavior.
OBJ-05	Web monitoring	Module B	Live dashboard with feed controls, KPIs, and stream views.
OBJ-06	Automation integration	Module C	Webhook-based alert routing through n8n and Telegram.

backend, web, and automation delivery tracks to maintain a consistent production-oriented scope.

1.3.7 Target Audience

The system targets three stakeholder groups with complementary goals:

- **Administrators:** manage manager accounts and access governance.
- **Managers:** configure operational settings, monitor live queue status, react to alerts, and use indicators for staffing and service-quality planning.
- **End users:** benefit indirectly from reduced waiting times and smoother service flow.

This separation is important because each group consumes different levels of abstraction, from per-feed live events to aggregated performance trends.

1.4 Project Management Methodology

1.4.1 Choice of Methodology: Agile

An Agile approach was selected to deliver functional increments quickly while reducing integration risks early. The project combines multiple technical domains (computer vision, queue analytics, backend APIs, frontend workflows, and automation), each with uncertainties that are difficult to resolve using a single monolithic delivery cycle.

Iterative delivery made it possible to validate contracts progressively: first perception outputs, then analytical reliability, then API/runtime integration, then web usability, and finally workflow automation and hardening.

1.4.2 SCRUM

Development followed sprint-based Scrum planning with iterative validation. For report readability and implementation traceability, work is grouped into three logical modules.

1.4.2.1 Module A: Backend Core (Sprints 1-3)

This module established the vision and analytics backbone:

- Sprint 1: detection, tracking, and zone filtering foundation,
- Sprint 2: queue metrics and stability handling,
- Sprint 3: camera connectivity and backend API exposure.

1.4.2.2 Module B: Web Experience (Sprints 4-5)

This module focused on the web supervision experience:

- Sprint 4: web design system and frontend scaffolding,

- Sprint 5: frontend integration with live backend metrics.

1.4.2.3 Module C: Automation and Finalization (Sprints 6-8)

This module covered integration hardening and closure:

- Sprint 6: n8n automation and Telegram alerting,
- Sprint 7: evaluation and optimization,
- Sprint 8: report finalization and project consolidation.

Table 1.5: Scrum module mapping used in this report

Module	Sprints	Main focus	Validation perspective
Module A: Backend Core	1 to 3	Vision pipeline, queue analytics, camera connectivity, API services	Technical correctness of perception and metrics contracts.
Module B: Web Experience	4 to 5	Frontend design system, dashboard scaffolding, live integration	Usability, interaction continuity, and live data rendering stability.
Module C: Automation and Finalization	6 to 8	n8n alerting, optimization, validation, and reporting	End-to-end operability, alert reliability, and delivery readiness.

1.5 Conclusion

This chapter positioned the project in its organizational and technical context, formalized the addressed operational problem, and justified the selected delivery strategy. Chapter

2 builds on this baseline by formalizing requirements, architecture constraints, and analysis artifacts used to guide implementation decisions.

Table of Content

2.1	Introduction	13
2.2	Requirements Specification	13
2.2.1	Identification of Actors	13
2.2.2	Functional Requirements	13
2.2.3	Non-functional Requirements	14
2.3	Product Backlog	15
2.4	Global Use Case Diagram	17
2.5	Sprint Scope Overview	17
2.6	Data Modeling	18
2.6.1	Core Entities	18
2.6.2	Data Lifecycle and Persistence Strategy	18
2.7	Project Environment	20
2.7.1	Hardware and Software Environment	20
2.7.2	Technologies Used	21
2.8	Software Architecture	22
2.8.1	Overall Architecture	22
2.8.2	Backend Architecture	23
2.8.3	Frontend Architecture	23
2.9	Conclusion	23

2.1 Introduction

This chapter defines the project requirements and shows how those requirements shaped the architecture choices that follow. Instead of listing features in isolation, it ties the functional scope, actors, and layers back to the implementation plan used across the sprint chapters.

The chapter goes beyond a feature list. It sets out requirement traceability, actor responsibilities, quality constraints, sprint-scope mapping, core data entities, and the architecture layers used throughout the report.

2.2 Requirements Specification

2.2.1 Identification of Actors

The system involves two primary actors:

- **Administrator:** manages manager accounts (create, update, delete) and controls access governance.
- **Manager:** configures sources and thresholds, monitors live queues, reacts to alerts, and reviews performance indicators.

Although these roles interact with the same platform, they consume different abstractions. Administrators require account-governance visibility, while managers require operational depth from setup through live supervision and KPI interpretation.

2.2.2 Functional Requirements

The system must provide the following core functionalities:

- real-time person detection and tracking,
- configurable queue zone definition,

Table 2.1: Actor responsibility matrix

Actor	Main responsibilities	Typical interactions
Administrator	Create, update, and delete manager accounts; enforce access governance rules.	Uses account-management and audit-oriented flows to maintain authorized manager access.
Manager	Configure feeds, zones, and thresholds; monitor live queue status; react to warning/critical alerts; review trends and performance summaries.	Uses setup workflows, dashboard KPIs, live feed views, alert panels, and analytics views for operational decision-making.

- estimation of arrival rate (λ), service rate (μ), and wait time (W),
- queue stability classification with robust wait-time fallback behavior,
- real-time web dashboard updates and alerts,
- RTSP and ONVIF camera integration,
- automated webhook and Telegram notifications,
- historical data logging and analytics.

To support verification and sprint-level accountability, these requirements are formalized in a traceability matrix.

2.2.3 Non-functional Requirements

The main quality constraints are:

- **Performance:** maintain real-time behavior and low alert latency.
- **Scalability:** support multiple feeds and independent feed lifecycle control.
- **Reliability:** tolerate transient source failures and integration interruptions.
- **Usability:** provide clear web workflows for monitoring and configuration.
- **Security:** protect integration channels and service endpoints.

Table 2.2: Functional requirement traceability matrix

ID	Requirement statement	Priority	Verification strategy
FR-01	Detect people from live or recorded video streams in near real time.	High	Validate detector outputs and runtime overlays in the perception layer.
FR-02	Maintain identity continuity through tracking for event-level queue analysis.	High	Validate tracker ID continuity in representative runtime scenarios.
FR-03	Restrict counting to configurable polygon queue zones.	High	Validate polygon inclusion behavior in the zone-management layer.
FR-04	Compute λ , μ , and expected wait time from event windows.	High	Validate queue formulas and stability handling in the analytics layer.
FR-05	Classify stable/unstable queue states and apply safe fallback wait-time logic.	High	Validate stability branches and fallback behavior in queue analytics.
FR-06	Expose feed control and live updates through API and websocket channels.	High	Validate endpoint and runtime behavior through API and integration tests.
FR-07	Dispatch alert payloads to external automation workflows.	Medium	Validate webhook dispatch and n8n branch behavior using script-based scenarios.
FR-08	Persist operational metrics and metadata for analysis and auditing.	Medium	Validate SQLite persistence behavior through runtime execution and database checks.

- **Maintainability:** keep modular code organization and testable interfaces.

These quality attributes are treated as engineering contracts, not only design intentions.

2.3 Product Backlog

The product backlog was prioritized by business value and technical dependency. For implementation governance, backlog items were grouped into three domains:

- **Backend domain:** perception pipeline, queue analytics, API and stream interfaces.

Table 2.3: Non-functional requirement quality matrix

ID	Quality attribute	Target indicator	Validation approach
NFR-01	Performance	Stable near-real-time loop and bounded alert propagation delay.	Runtime observation under representative feed scenarios.
NFR-02	Scalability	Independent lifecycle control for multiple feed contexts.	Feed registry behavior and state-transition verification.
NFR-03	Reliability	Graceful handling of camera drops and webhook transmission failures.	Reconnect paths and retry/fallback test scenarios.
NFR-04	Usability	Clear setup-to-monitoring flow for non-developer users.	Dashboard integration tests and workflow walkthroughs.
NFR-05	Security	Controlled integration access via shared secret and credential isolation.	Webhook secret validation and credentialized channel checks.
NFR-06	Maintainability	Modular architecture with explicit contracts and targeted tests.	Code organization review and test-suite coverage by module.

- **Web domain:** dashboard UX, live monitoring integration, zone editing workflows.
- **Automation domain:** n8n workflow integration, alert routing, webhook reliability.

Early execution emphasized backend foundations to reduce downstream integration risk. This sequencing ensured that web and automation layers consumed a stable metric contract rather than moving targets.

Table 2.4: Backlog domain structuring

Domain	Representative backlog scope	Dependency rationale
Backend	Detection/tracking, zone logic, queue formulas, and feed runtime services.	Provides source-of-truth metrics consumed by all upper layers.
Web	Dashboard pages, typed API integration, websocket synchronization, feed controls.	Depends on stable backend contracts and event schemas.
Automation	Webhook payload contract, routing, cooldown, external notifications, archival.	Depends on reliable alert semantics and validated thresholds.

2.4 Global Use Case Diagram

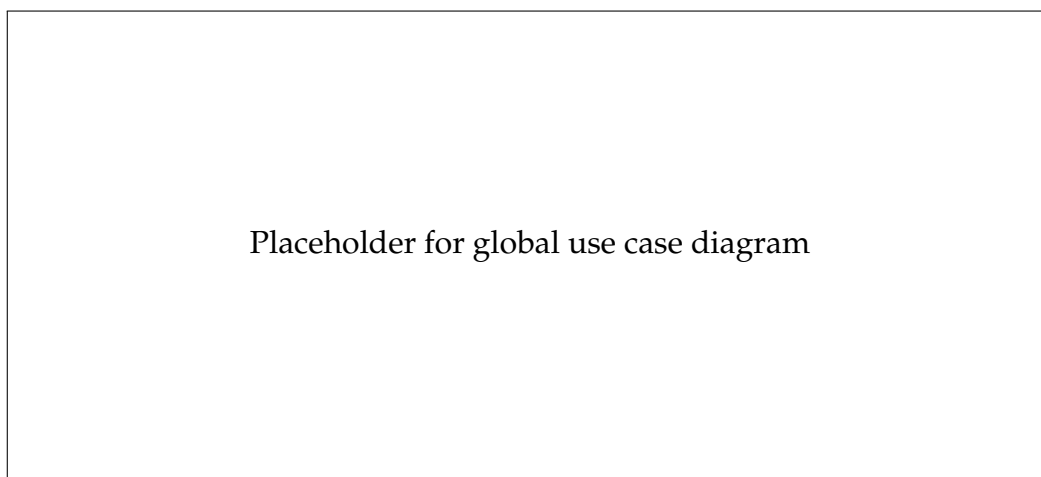


Figure 2.1: Global use case diagram of the queue wait-time estimation system.

At system level, use cases can be organized into four interaction groups: configuration, monitoring, analytics interpretation, and alert handling. The global diagram should reflect these groups and their actor responsibilities.

2.5 Sprint Scope Overview

The project implementation is organized into eight sprints. Table 2.6 summarizes sprint scope and expected outcomes without using calendar-based timeline tracking.

Table 2.5: Primary global use cases

Use case ID	Use case summary	Primary actor
UC-01	Create, update, or remove manager accounts.	Administrator
UC-02	Register and configure a video feed source.	Manager
UC-03	Define or update queue zone geometry.	Manager
UC-04	Monitor live queue metrics and feed status.	Manager
UC-05	Receive and interpret warning or critical alerts.	Manager
UC-06	Review trends and performance indicators.	Manager

2.6 Data Modeling

The data model combines runtime monitoring entities and persistence entities. Its design objective is to support both low-latency supervision and post-hoc analysis with minimal contract drift across modules.

2.6.1 Core Entities

The main objects are:

- **VideoFeed**: source identity, connectivity, runtime status, and configuration.
- **FeedTransportCapabilities**: per-feed readiness flags for WebRTC and MJPEG routing.
- **QueueMetrics**: timestamped indicators (people in zone, λ , μ , W , stability).
- **Alert**: severity, threshold context, and notification payload.
- **Establishment** and **Cashier**: business metadata used to organize feeds.

2.6.2 Data Lifecycle and Persistence Strategy

Runtime metrics are generated continuously and distributed through live channels, while selected records are persisted for auditability and trend analysis. This dual-path

Table 2.6: Project sprint scope overview

Sprint	Main outcome	Representative validation lens
Sprint 1	Detection, tracking, and queue-zone perception foundation	Perception correctness and tracker continuity
Sprint 2	Queue analytics and stability handling	Formula validity and stability-branch consistency
Sprint 3	Camera connectivity and backend API exposure	Runtime control and contract stability
Sprint 4	Web design system and frontend scaffolding	UX structure and component baseline
Sprint 5	Frontend integration and live monitoring	API/websocket synchronization quality
Sprint 6	n8n automation and Telegram alerting	Alert routing and delivery reliability
Sprint 7	Evaluation and optimization	Test depth and runtime hardening
Sprint 8	Final consolidation, reporting, and delivery	End-to-end consistency and documentation quality

strategy (stream plus persistence) improves resilience: temporary integration failures do not imply complete data loss.

SQLite database persistence stores operational metrics, structured metadata, and alert archives in one source of truth. This supports both operational continuity and post-run diagnosis.

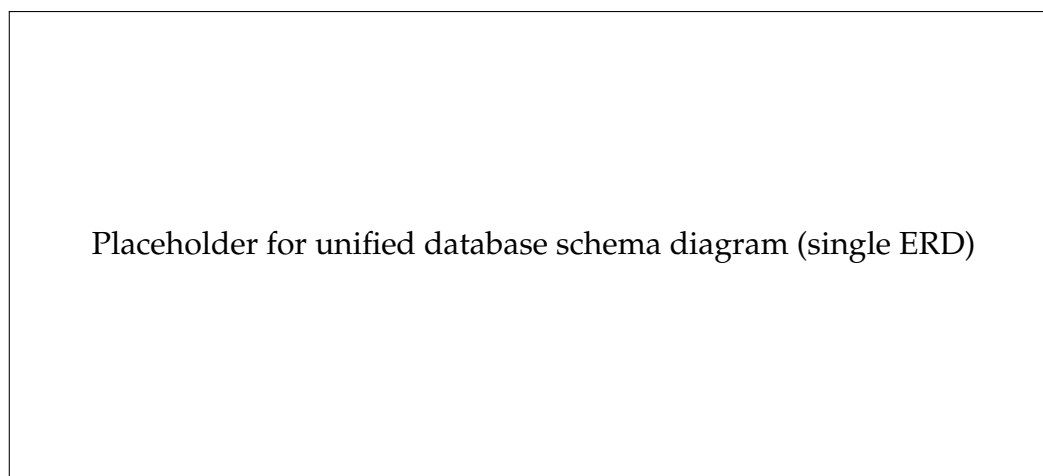


Figure 2.2: Unified SQLite schema used for feed metadata, queue sessions, and alert archival.

Table 2.7: Conceptual entity mapping

Entity	Core semantics	Producer/consumer context
VideoFeed	Declares source type, identity, status, and runtime control metadata.	Produced by onboarding/configuration flows; consumed by runtime and dashboard layers.
FeedTransportCapabilities	Declares transport readiness (WebRTC and MJPEG), source mode, and unavailability reason for each feed.	Produced by API runtime transport endpoint; consumed by dashboard playback routing and fallback logic.
QueueMetrics	Captures analytical outputs used for monitoring and decision support.	Produced by analysis loop; consumed by websocket/API/dashboard and alert logic.
Alert	Encodes threshold breach context and notification-ready payload content.	Produced by threshold evaluation; consumed by webhook automation and archival paths.
Establishment / Cashier	Binds feeds to business-level organization context.	Produced by metadata management APIs; consumed by setup and filtering interfaces.

Figure 2.2 is intentionally kept as a single consolidated database view to avoid duplication across sprint chapters.

2.7 Project Environment

2.7.1 Hardware and Software Environment

The development workflow combines a Python backend environment, a TypeScript frontend environment, and containerized automation services. The project is designed to run on standard workstations and can exploit hardware acceleration when compatible GPU resources are available.

Table 2.8: Execution environment profile

Environment layer	Main tooling	Purpose
Backend runtime	Python ecosystem with vision and scientific libraries	Perception, queue analytics, and integration dispatch.
Frontend runtime	Node/Bun-based TypeScript toolchain with Vite	Dashboard rendering, interaction workflows, and live-state synchronization.
Automation runtime	Self-hosted n8n with webhook endpoints	Alert routing, cooldown logic, and external notifications.
Persistence/runtime support	SQLite database store (metrics, metadata, alerts)	Historical analysis and system state continuity.

2.7.2 Technologies Used

The selected technology stack is:

- **Vision pipeline:** YOLO26, ByteTrack, OpenCV [2], [3], [4].
- **Queue analytics:** NumPy, Pandas, SciPy, M/M/1 modeling [1].
- **Backend API:** FastAPI and Pydantic v2.
- **Live transport:** WebRTC signaling with MediaMTX WHEP gateway and MJPEG fallback [5], [6].
- **Frontend:** React, TypeScript, Vite, Tailwind CSS.
- **Automation:** n8n workflows with Telegram notifications.
- **Persistence:** SQLite database for metrics, metadata, and alert archival.

The stack was chosen to keep each layer modular: every component needed a clear boundary, practical interoperability with the next layer, and enough flexibility to support live queue monitoring.

2.8 Software Architecture

2.8.1 Overall Architecture

The architecture is easier to read as four layers:

1. **Video acquisition:** local files, RTSP streams, and ONVIF discovery.
2. **Perception:** detection, tracking, and queue-zone filtering.
3. **Analysis:** queue metrics estimation and stability handling.
4. **Output and integration:** web dashboard, webhook dispatch, and alert automation.

That separation keeps perception, analysis, UI, and automation independent enough to test on their own. For example, queue formulas depend on perception events, but do not depend on frontend rendering details.

Table 2.9: Architecture layer-to-module mapping

Layer	Responsibilities	Representative components
Video acquisition	Source opening, stream transport handling, reconnect behavior.	Source-access and camera-adaptor layers
Perception	Person detection, tracking continuity, zone-aware filtering.	Detection, tracking, and zone-management layers
Analysis	Event-window rates, wait-time logic, and stability-aware fallback handling.	Queue analysis and stability-handling logic
Output/integration	API/websocket exposure, webhook dispatch, persistence, dashboard consumption.	API runtime, webhook delivery, and frontend integration layers

2.8.2 Backend Architecture

The backend exposes REST and websocket interfaces for feed lifecycle control and live metrics. It includes modules for detector orchestration, tracking, queue analysis, threshold detection, webhook delivery, and camera-source management.

Runtime orchestration focuses on feed-level lifecycle control and event broadcasting. That keeps feed state transitions independent and gives the dashboard and automation pipeline a clear event contract.

Persistence and integration are treated as part of the core system, not add-ons: SQLite-backed state and webhook reliability both belong to the operational architecture.

2.8.3 Frontend Architecture

The frontend is built as a component-based interface with typed API integration, feed-oriented dashboard composition, and websocket-driven synchronization. Data access lives in hooks and client utilities, while pages and reusable components handle presentation.

That split keeps API changes localized to the integration layer and leaves the visual components focused on rendering and interaction.

2.9 Conclusion

This chapter defines the functional baseline and the architectural logic that the later sprint chapters realize in code and tests. The next chapters move from perception foundations to connectivity, automation, and evaluation.

Sprint 1: Foundation and Perception

Table of Content

3.1	Introduction	25
3.2	Sprint Backlog	25
3.3	Sprint Analysis	26
3.3.1	Detailed Use Case Diagram	26
3.3.2	Use Case Scenarios	26
3.4	Design	27
3.4.1	Activity Diagram	27
3.4.2	Sequence Diagrams	27
3.5	Implementation	28
3.5.1	Detection Layer	28
3.5.2	Tracking Layer	28
3.5.3	Zone-Aware Filtering	28
3.5.4	Main Loop Integration	28
3.6	Conclusion	29

3.1 Introduction

Sprint 1 built the perception pipeline that turns video frames into queue-relevant events. The main outcome was a reliable chain from capture to detection, tracking, and zone filtering inside the frame loop.

3.2 Sprint Backlog

The committed Sprint 1 backlog focused on the minimum viable perception chain:

- Integrate YOLO-based person detection with configurable model size and inference device.
- Integrate persistent tracking so each detected person keeps a stable ID across frames.
- Define queue zones as polygons and count only detections inside these zones.
- Build a real-time loop that connects capture, detection, tracking, zone filtering, and rendering.
- Expose runtime controls through CLI options for source type, model parameters, and processing stride.

3.3 Sprint Analysis

3.3.1 Detailed Use Case Diagram

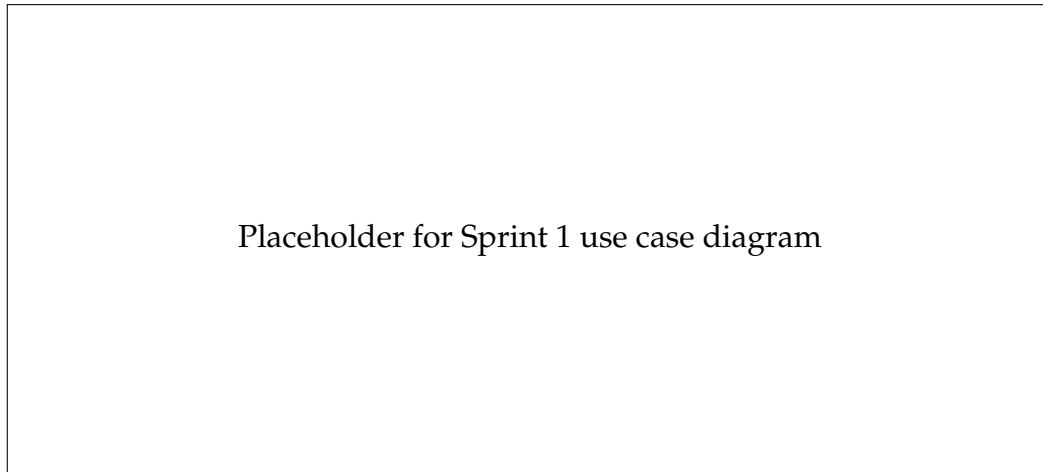


Figure 3.1: Sprint 1 use case diagram for detection and tracking in queue zones.

The use case follows a manager configuring a source and queue zone, then letting the runtime turn each frame into queue state updates. That sequence becomes the base for the analytics and automation layers that follow.

3.3.2 Use Case Scenarios

Three key scenarios are implemented in Sprint 1:

1. **Person entry into queue zone:** a new tracker ID appears inside the polygon and is treated as a queue arrival candidate.
2. **Tracker continuity across frames:** the same person is followed through consecutive frames using ByteTrack, reducing duplicate counting.
3. **Zone filtering under mixed scenes:** detections outside the polygon are ignored for queue metrics, preventing unrelated people from polluting queue state.

3.4 Design

3.4.1 Activity Diagram

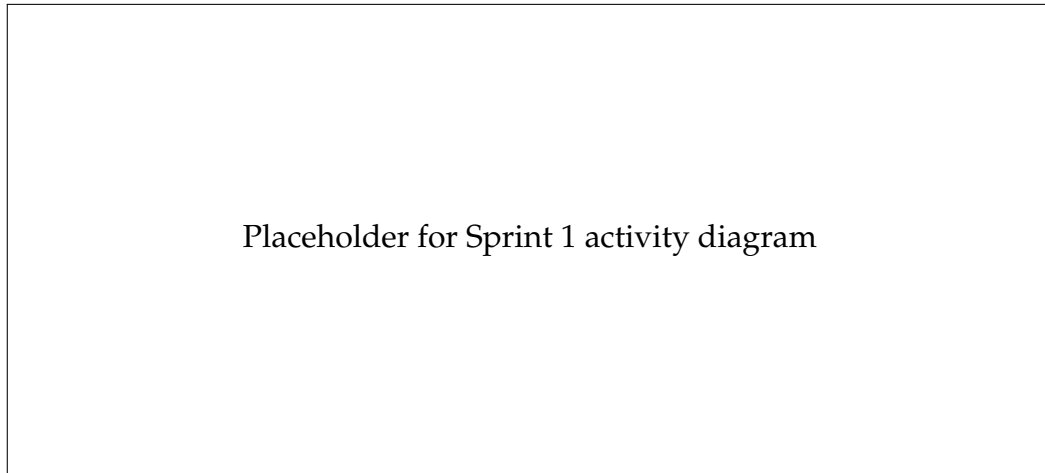


Figure 3.2: Sprint 1 activity flow from frame ingestion to queue-zone updates.

The activity starts from a video source (local file, webcam, or RTSP stream), applies person detection, enriches detections with tracker IDs, then evaluates polygon-zone membership. The filtered state is forwarded to the queue analyzer for rate and wait-time computation.

3.4.2 Sequence Diagrams

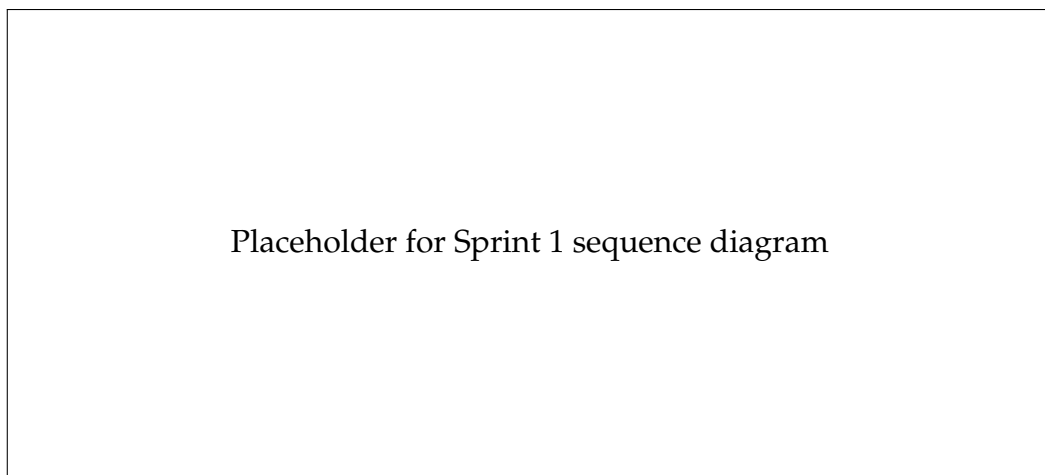


Figure 3.3: Sprint 1 sequence diagram from video ingestion to queue-state updates.

The implemented sequence in the codebase is: `VideoStream/RTSPCamera → PersonDetector.detect() → ObjectTracker.update() → ZoneManager.trigger() → QueueAnalyzer.update()`.

This call chain is orchestrated per frame by the runtime `run()` loop.

3.5 Implementation

3.5.1 Detection Layer

The detection layer is built around the `PersonDetector` class. It wraps Ultralytics YOLO inference and filters results to person class only (COCO class ID 0). The detector resolves device selection (`auto/cpu/cuda/mps`) and supports configurable image size for throughput control [2].

3.5.2 Tracking Layer

The tracking layer uses `supervision.ByteTrack`. The tracker propagates persistent `tracker_id` values across frames, which is essential for counting arrivals and departures rather than raw per-frame detections [3].

3.5.3 Zone-Aware Filtering

Zone logic is handled by the zone-management module. It accepts polygons in normalized or pixel coordinates, auto-scales normalized points to frame size, and evaluates inclusion using `PolygonZone` with bottom-center anchors for robust queue-ground contact checks.

3.5.4 Main Loop Integration

The pipeline integration is handled in the runtime orchestration layer. The runtime supports:

- Source abstraction via `open_video_source()` for file/webcam/RTSP.
- Process stride control through `-process-every-n-frames`.
- Real-time overlay rendering (detections, IDs, zone, and metrics).
- Headless mode and dashboard event emission hooks for future API/runtime integration.

The sprint left behind a stable perception baseline that later sprints could build on for queue analytics and automation integration [4].

3.6 Conclusion

Sprint 1 delivered the operational perception core: reliable person detection, persistent IDs, and queue-zone filtering. That baseline gave Sprint 2 stable event data to turn into rates and wait-time estimates.

Sprint 2: Analytics and Uncertainty

Table of Content

4.1	Introduction	31
4.2	Sprint Backlog	31
4.3	Design	31
4.3.1	Component Diagram	31
4.3.2	Data Flow Diagram	32
4.3.3	Sequence Diagrams	32
4.4	Implementation	33
4.4.1	Queue Event Windows and Rate Estimation	33
4.4.2	Wait-Time Logic and Stability Handling	33
4.4.3	Runtime Contract for Monitoring and Automation	34
4.5	Conclusion	34

4.1 Introduction

Sprint 2 shifted the system from counting people to describing queue dynamics. The analytical layer now derives arrival and service rates, computes wait time, and marks when the queue should be treated as unstable.

4.2 Sprint Backlog

Sprint 2 backlog priorities were:

- Implement event-based queue rate estimation from tracked IDs in zone.
- Compute wait-time estimates with explicit stable/unstable queue handling.
- Add denominator-safety guards and fallback wait-time logic.
- Expose a consistent runtime metric contract for dashboard and webhook consumption.

4.3 Design

4.3.1 Component Diagram

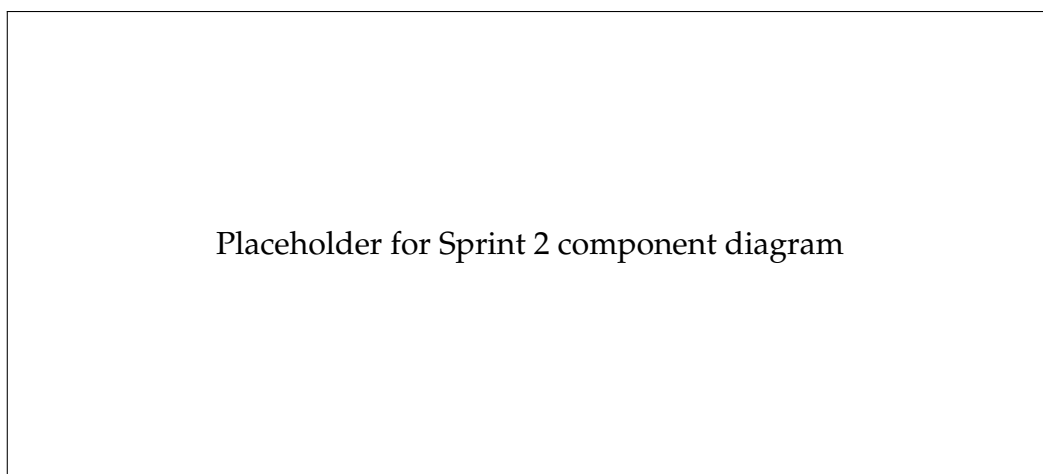


Figure 4.1: Sprint 2 components for queue analysis and stability handling.

At component level, QueueAnalyzer receives in-zone tracker IDs and produces QueueMetrics consumed by logs, webhook payloads, and dashboard updates.

4.3.2 Data Flow Diagram

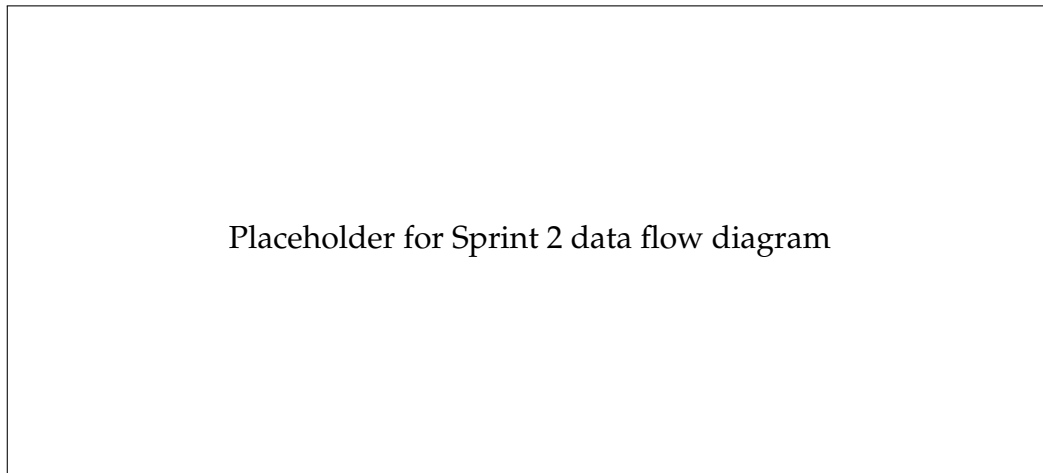


Figure 4.2: Data flow from tracked events to queue metrics and stability state.

Data flow is event-centric: per-frame ID transitions produce arrival/departure events, event windows produce rates (λ, μ) , and rates with occupancy produce wait-time estimates plus stable/unstable state labels.

4.3.3 Sequence Diagrams

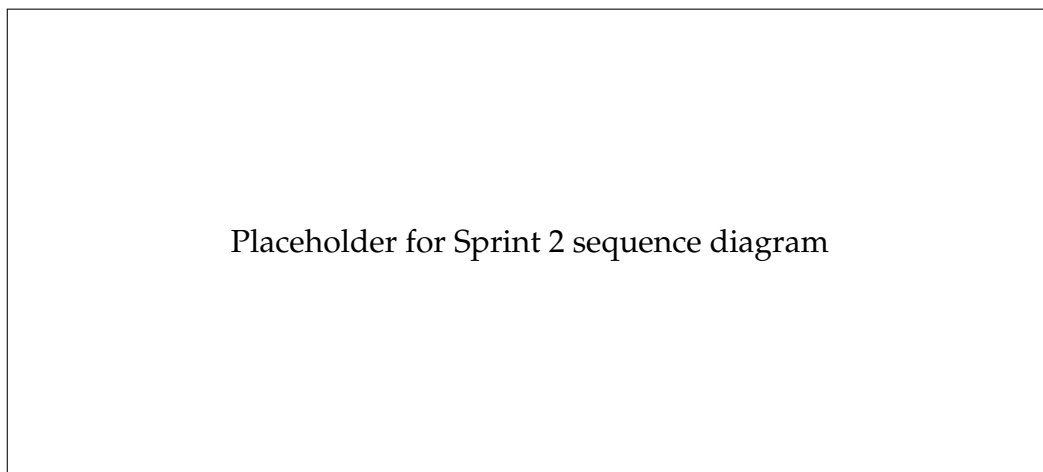


Figure 4.3: Sprint 2 sequence diagram for queue rate estimation and stability handling.

The implemented sequence is: `QueueAnalyzer.update()` computes counts and rates, evaluates queue stability, and applies either the stable wait-time formula or fallback wait-time branch before publishing `QueueMetrics`.

4.4 Implementation

4.4.1 Queue Event Windows and Rate Estimation

In the queue-analysis module, arrivals and departures are tracked as timestamp dequeues over sliding windows. Let N_a and N_d be events in windows T_a and T_d :

$$\lambda = \frac{N_a}{T_a}, \quad \mu = \frac{N_d}{T_d}$$

The implementation enforces a minimum event count before returning non-zero rates, reducing unstable early-window estimates.

4.4.2 Wait-Time Logic and Stability Handling

The runtime marks queue stability with:

$$\text{queue_stable} = (\lambda < \mu) \wedge (\mu > 0)$$

The wait-time logic follows two branches:

- Stable queue: $W = \frac{1}{\mu - \lambda}$
- Unstable but service observed: fallback $W = \frac{L}{\mu}$, where L is people in zone

When no service events are observed, the implementation currently returns a zero fallback.

4.4.3 Runtime Contract for Monitoring and Automation

The final `QueueMetrics` object includes:

- point estimates (λ, μ, W) ,
- people-in-zone occupancy,
- stability flag for safe downstream interpretation.

These fields are propagated to logs, overlays, webhook payloads, and dashboard events.

4.5 Conclusion

Sprint 2 delivered the analytical core of the project: event-to-rate modeling, stable and unstable wait-time semantics, and a runtime metric contract that downstream components can consume without guessing.

Table of Content

5.1	Introduction	36
5.2	Sprint Backlog	36
5.3	Design	37
5.3.1	Component Diagram	37
5.3.2	Data Flow Diagram	37
5.3.3	Sequence Diagrams	38
5.4	Implementation	38
5.4.1	RTSP Source Integration	38
5.4.2	ONVIF Discovery and Stream Resolution	39
5.4.3	FastAPI BFF and Typed Contract	39
5.4.4	Transport Capability and WebRTC Signaling	39
5.4.5	Runtime Orchestration and Real-Time Events	40
5.4.6	Persistence and Recovery	41
5.5	Conclusion	41

5.1 Introduction

Sprint 3 turned the single local pipeline into a multi-source platform. Camera onboarding, feed lifecycle control, and real-time delivery now sit behind a typed API and websocket contract.

5.2 Sprint Backlog

The Sprint 3 backlog addressed four integration tracks:

- RTSP connectivity with authentication, transport control, reconnection, and snapshots.
- ONVIF discovery and stream resolution for IP camera onboarding.
- REST API endpoints for feed CRUD, metadata management, and diagnostics.
- Real-time delivery through websocket events and tiered video transport (WebRTC primary, MJPEG fallback, snapshots for zone workflows).

5.3 Design

5.3.1 Component Diagram

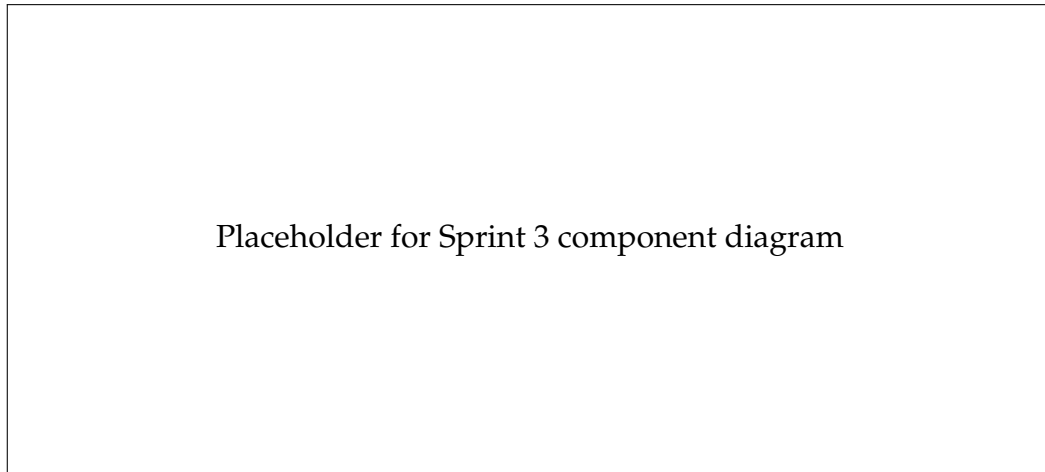


Figure 5.1: Sprint 3 components for camera integration and backend services.

The design keeps camera adapters, API models, runtime feed orchestration, and web-socket broadcasting separate so each part can evolve without dragging the others with it.

5.3.2 Data Flow Diagram

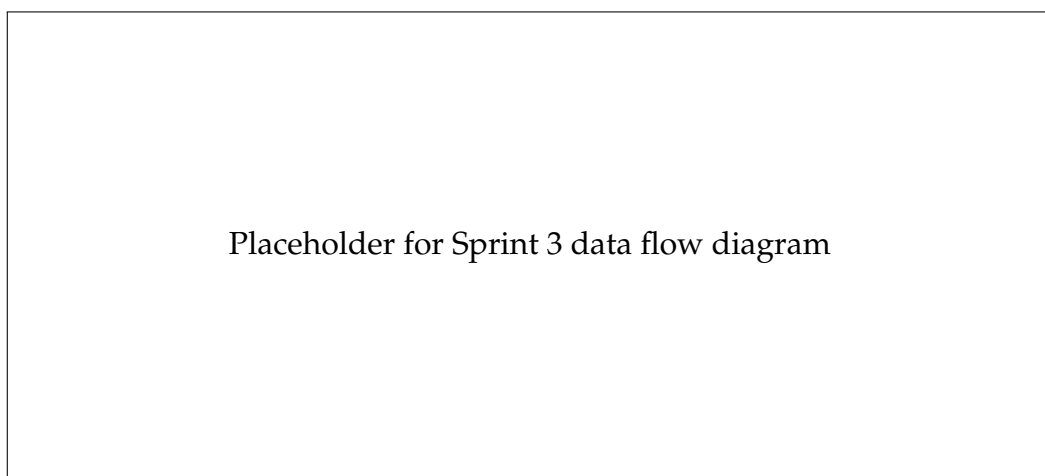


Figure 5.2: Data flow from video feeds to API responses and WebSocket broadcasts.

The principal data paths are: (1) onboarding/test flows through REST endpoints, (2) feed-control flows through start/stop/restart operations, and (3) runtime event flows from worker JSONL streams to websocket broadcast and dashboard cache updates.

5.3.3 Sequence Diagrams

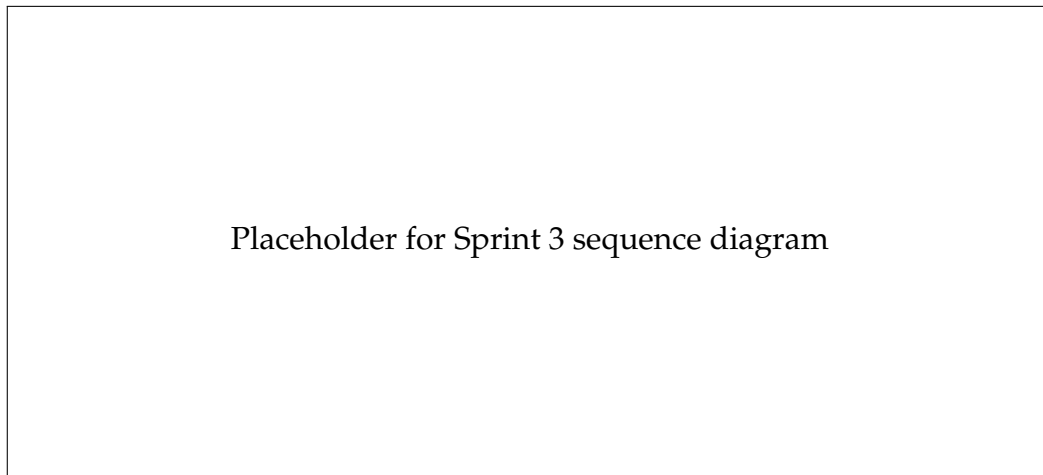


Figure 5.3: Sprint 3 sequence diagram for feed orchestration and live event distribution.

The implemented sequence for a running feed is: frontend request $\rightarrow$ `/api/feeds/*` $\rightarrow$ FeedRegistry operation $\rightarrow$ worker process lifecycle change $\rightarrow$ websocket `feed_status/metrics_update` event.

5.4 Implementation

5.4.1 RTSP Source Integration

In the RTSP camera adapter, the `RTSPCamera` class supports:

- credential embedding with safe log redaction,
- TCP/UDP transport selection through OpenCV/FFMPEG options,
- connection testing and snapshot capture,
- automatic reconnection with bounded retries.

The source abstraction layer routes local/webcam sources to `VideoStream` and RTSP URLs to `RTSPCamera`.

5.4.2 ONVIF Discovery and Stream Resolution

The ONVIF client module implements WS-Discovery probes, `ProbeMatch` parsing, capability extraction, media-service resolution, and RTSP URI retrieval through SOAP requests with digest/basic authentication fallback. This enables dynamic IP camera onboarding without hardcoding stream URLs.

5.4.3 FastAPI BFF and Typed Contract

The API layer is implemented as a FastAPI backend-for-frontend, with typed request and response models. Major endpoint groups include:

- feed management (create/list/start/stop/restart/delete),
- source onboarding (RTSP test/snapshot, ONVIF discovery/streams/test),
- metadata (establishments and cashiers),
- integration and health (webhook status/test, system health),
- live transport channels (websocket, transport capability endpoint, WebRTC signaling endpoint, and MJPEG stream endpoint).

5.4.4 Transport Capability and WebRTC Signaling

To reduce end-to-end viewing latency while preserving backward compatibility, Sprint 3 now exposes a capability-driven transport contract aligned with WebRTC signaling standards [5], [7].

The endpoint `/api/feeds/{feed_id}/transport` returns per-feed `FeedTransportCapabilities` with both `webrtc` and `mjpeg` readiness fields. The frontend consumes this contract to

decide transport routing:

- `enabled`: feature-level availability,
- `ready`: runtime readiness for immediate use,
- `source_mode`: `direct`, `annotated`, or `none`,
- `reason`: machine-readable explanation when transport is unavailable.

When WebRTC is ready, the endpoint `/api/feeds/{feed_id}/webrtc/offer` accepts a browser SDP offer and proxies it to MediaMTX WHEP, then returns the SDP answer [6]. The backend validates feed existence, running state, and RTSP-source eligibility before proxying.

Operational status semantics are explicit:

- 200: offer accepted and SDP answer returned,
- 404: feed not found,
- 409: feed not running, unsupported source, or transport not ready,
- 422: invalid SDP payload,
- 502/503: upstream or service-availability failures.

In `direct` mode, backend runtime can ensure MediaMTX path configuration through the control API before offer proxying. Environment-driven configuration (WHEP URL, control URL, timeout, and feature flag) keeps deployment adaptable without changing API contracts.

5.4.5 Runtime Orchestration and Real-Time Events

Runtime control sits in the API orchestration service. `FeedRegistry` manages feed state, persistence sync, worker process lifecycle, and event dispatch. The worker command launches the headless analysis runtime and writes event JSONL. `WebSocketHub` broad-

casts standardized events: `snapshot`, `feed_status`, `metrics_update`, `alert_fired`, and `system_warning`.

For video surfaces, `FeedFrameStreamManager` supports shared MJPEG buffering with capture-to-worker handoff continuity, preventing stream discontinuities when runtime mode changes.

5.4.6 Persistence and Recovery

The persistence layer provides SQLite storage for feed configuration, metadata, video sessions, and archived alerts. On API startup, persisted feeds are restored and stale running sessions are safely recovered to a stopped state.

5.5 Conclusion

By the end of Sprint 3, the backend could onboard cameras, manage feed state, and broadcast events to the frontend without hardcoding source details. That made the web UI integration in Sprint 4 much more direct.

Sprint 4: Web Design and Scaffolding

Table of Content

6.1	Introduction	43
6.2	Sprint Backlog	43
6.3	Design	43
6.3.1	UI and UX Design	43
6.3.2	Frontend Architecture	44
6.3.3	Design Rationale	44
6.4	Implementation	45
6.5	Conclusion	45

6.1 Introduction

Sprint 4 established the visual system and page structure for the dashboard. The main deliverable was a frontend scaffold that matched the monitoring workflow before live data wiring was added.

6.2 Sprint Backlog

Sprint 4 backlog focused on design-to-code readiness:

- Define a consistent visual design system (colors, typography, spacing, component rules).
- Produce complete page layouts for onboarding, monitoring, editing, analytics, and settings.
- Scaffold a React + TypeScript frontend project with reusable UI primitives.
- Establish routing and layout composition patterns for future live integration.

6.3 Design

6.3.1 UI and UX Design

The project design specification defines a “Terminal Minimal” style system and a seven-page navigation model. The design language emphasizes:

- dark operational surfaces optimized for continuous monitoring,
- strong semantic colors for status and alert levels,
- card-based information density with consistent spacing and hierarchy,
- explicit workflow guidance in setup and zone-editing screens.

The designed page set includes landing, dashboard, setup wizard, zone editor, analytics, settings, and heatmap views, which directly informed component boundaries in the frontend implementation.

6.3.2 Frontend Architecture

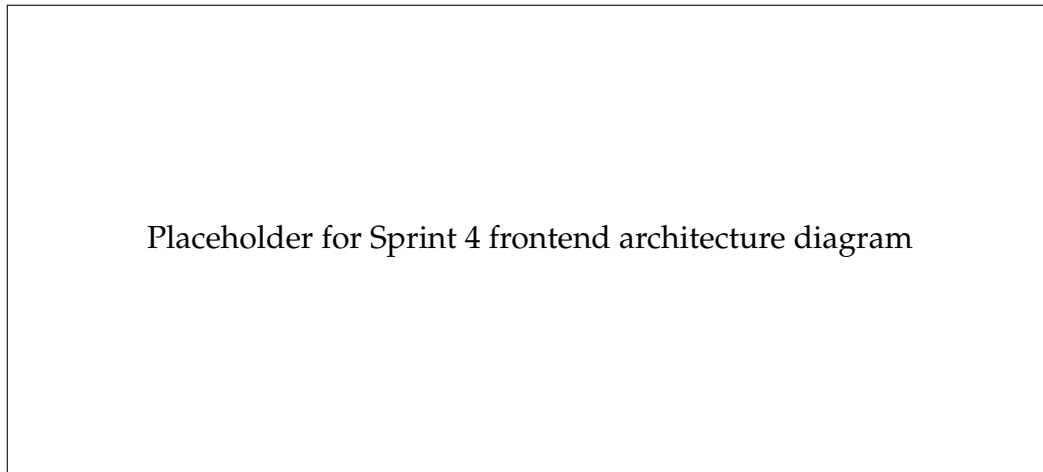


Figure 6.1: Frontend architecture for routing, components, and state boundaries.

The frontend scaffold separates page-level views, shared UI, hooks, and API adapters so the dashboard can grow without collapsing into one large component tree:

- page layer: route-level views (Dashboard, Analytics, ZoneEditor, Settings, etc.).
- component layer: reusable UI and domain panels.
- integration hook layer: stateful workflow and data hooks.
- API adapter layer: contract mapping and helper utilities.

6.3.3 Design Rationale

Sprint 4 was intentionally scoped as a scaffold sprint rather than a feature-complete UI release. The aim was to lock down navigation, component boundaries, and interaction patterns before live data and transport logic arrived in Sprint 5. That sequencing kept

the report shorter than the implementation-heavy sprints, but it also reflects the actual distribution of work and the role this sprint played in the overall delivery plan.

6.4 Implementation

Sprint 4 implementation established the frontend technical baseline:

- Vite + React + TypeScript project initialization with package-managed workspace configuration.
- Tailwind CSS and shadcn/Radix-driven UI composition.
- React Router page structure and layout shell integration.
- Initial dashboard and setup page scaffolding for subsequent live-data wiring.

The tradeoff in this sprint was deliberate: structure and design consistency came before real-time behavior. That made Sprint 5 easier because the route map and component boundaries were already settled.

6.5 Conclusion

Sprint 4 left the app with a clear route and component scaffold that reduced rework when live data arrived in Sprint 5.

Sprint 5: Frontend Integration and Live Monitoring

Table of Content

7.1	Introduction	47
7.2	Sprint Backlog	47
7.3	Design	48
7.3.1	Data Flow Diagram	48
7.3.2	Sequence Diagrams	48
7.4	Implementation	49
7.4.1	Typed API Contract Integration	49
7.4.2	Dashboard State Orchestration	49
7.4.3	WebSocket Reliability and Fallback Logic	49
7.4.4	Transport Capability Routing and WebRTC Lifecycle	50
7.4.5	Dashboard UI Integration	51
7.4.6	Integration Testing	51
7.5	Conclusion	51

7.1 Introduction

Sprint 5 linked the dashboard to live backend services and runtime events. With the API client, websocket updates, and transport routing in place, the frontend could behave as an operational control panel instead of a static shell.

7.2 Sprint Backlog

Sprint 5 backlog covered integration-critical items:

- implement a typed API client aligned with backend response contracts,
- integrate feed lifecycle actions (create/start/stop/restart/delete),
- integrate websocket events for low-latency dashboard updates,
- integrate capability-driven transport routing (WebRTC first, MJPEG fallback),
- implement resilient live monitoring behavior (reconnect and fallback),
- validate dashboard setup and onboarding workflows through integration tests.

7.3 Design

7.3.1 Data Flow Diagram

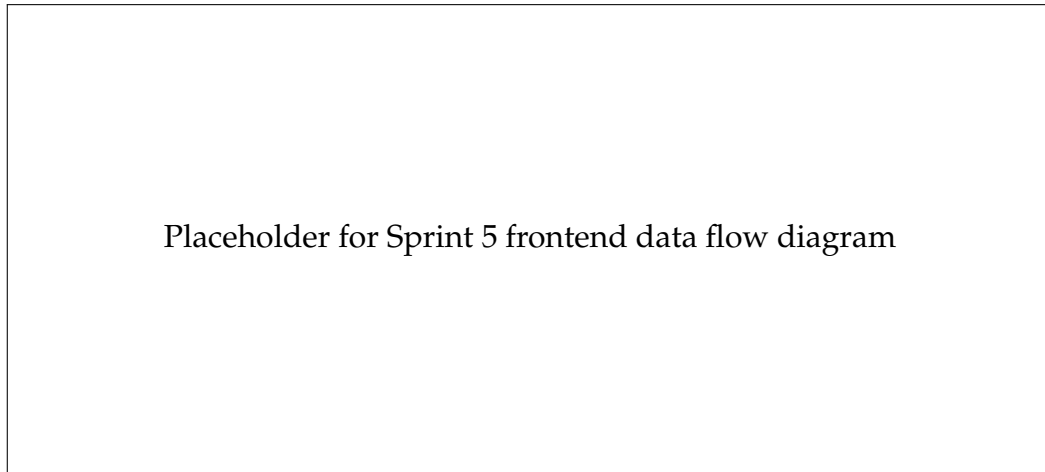


Figure 7.1: Data flow between backend services and frontend monitoring components.

In practice, REST endpoints handle CRUD and diagnostics, websocket events carry live updates, and local dashboard state aggregates those signals into KPI cards, activity logs, and attention panels.

7.3.2 Sequence Diagrams

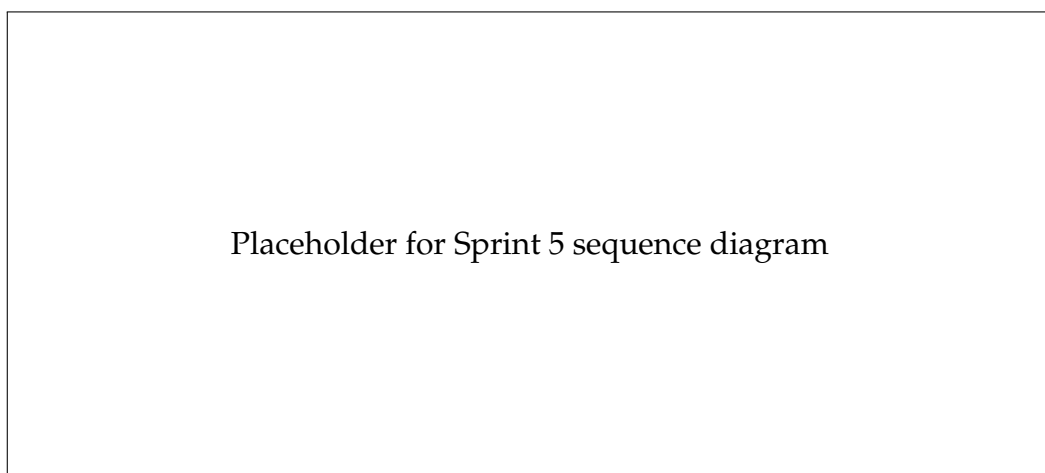


Figure 7.2: Sprint 5 sequence diagram for dashboard updates and alert surfacing.

Three primary sequences were implemented:

1. **Feed lifecycle:** dashboard action → REST endpoint → registry update → websocket `feed_status` event.
2. **Live metrics:** worker metrics event → backend websocket → frontend cache merge → visual refresh.
3. **Alert surfacing:** backend alert event → websocket `alert_fired/system_warning` → activity and attention views.

7.4 Implementation

7.4.1 Typed API Contract Integration

The frontend contract is implemented in a dedicated TypeScript API client layer. It defines typed entities for feeds, metrics, alerts, and websocket events, and wraps network calls with explicit error classes (`ApiError`, `NetworkError`, `ValidationError`). This improved reliability of frontend-side error handling and debugging.

7.4.2 Dashboard State Orchestration

A dedicated dashboard state hook coordinates data fetching and mutations through React Query. Derived indicators (online feeds, total people, average wait time, attention counts) are computed from normalized feed state and exposed to dashboard widgets.

7.4.3 WebSocket Reliability and Fallback Logic

The websocket coordination hook implements:

- reconnect backoff,
- heartbeat ping,

- stale-data watchdog,
- REST polling fallback during websocket degradation,
- batched metric flush to reduce UI re-render pressure.

When the connection drops, the dashboard falls back to REST polling instead of freezing, so operators still have a usable view during transient issues.

7.4.4 Transport Capability Routing and WebRTC Lifecycle

Live feed cards use capability-driven routing through the feed-grid component. For each running feed, the frontend queries `/api/feeds/{feed_id}/transport`. If `webrtc.enabled=true`, `webrtc.ready=true`, and `source_mode != none`, the dashboard selects WebRTC as the primary transport; otherwise it falls back to MJPEG.

The WebRTC lifecycle is implemented in a dedicated feed-connection hook using `RTCPeerConnection` [5]:

- create connection with a receive-only video transceiver,
- gather ICE candidates (bounded wait before signaling),
- send browser SDP offer to `/api/feeds/{feed_id}/webrtc/offer`,
- apply backend SDP answer and switch feed card to live state.

Reconnection uses exponential backoff (1s, 2s, 5s, 10s). Retry is intentionally skipped for non-recoverable API statuses (404, 409, 422). Under those conditions, the transport layer degrades to MJPEG while preserving dashboard continuity.

When backend-annotated mode is active on WebRTC, client-side detection overlays are suppressed to avoid duplicate box rendering on top of server-rendered frames.

7.4.5 Dashboard UI Integration

The dashboard page integrates live feed controls, setup flow dialogs, source onboarding (RTSP/ONVIF/local file), KPI visualizations, and chart components. The page composes domain hooks and reusable dashboard components into one operational interface.

7.4.6 Integration Testing

Frontend integration behavior is validated with Vitest and Testing Library through scenario-based suites that cover dashboard setup flow execution, websocket-event handling, WebRTC lifecycle and fallback logic, feed-grid transport routing, and API error-contract behavior.

7.5 Conclusion

By the end of Sprint 5, the dashboard could react to live state, switch transport modes, and fall back cleanly when websocket or WebRTC paths failed. Sprint 6 then used that live surface for alert automation.

Table of Content

8.1	Introduction	53
8.2	Sprint Backlog	53
8.3	Design	54
8.3.1	Component Diagram	54
8.3.2	Data Flow Diagram	54
8.4	Implementation	55
8.4.1	Webhook Payload Contract	55
8.4.2	Reliable Webhook Delivery	55
8.4.3	Runtime Alert Dispatch Integration	55
8.4.4	n8n Workflow Routing and Telegram Delivery	56
8.4.5	Automation Testing Artifacts	56
8.5	Conclusion	56

8.1 Introduction

Sprint 6 connected queue metrics to n8n so alerts could leave the backend in a controlled way. The focus was not just sending payloads, but keeping delivery reliable, auditable, and safe to repeat.

8.2 Sprint Backlog

Sprint 6 backlog focused on operational automation:

- define and validate a stable webhook payload contract,
- implement robust webhook transmission with retries and backup logging,
- integrate alert triggers from runtime threshold detection,
- implement n8n routing for authorization, cooldown, formatting, and delivery,
- provide test scripts for repeated workflow validation.

8.3 Design

8.3.1 Component Diagram

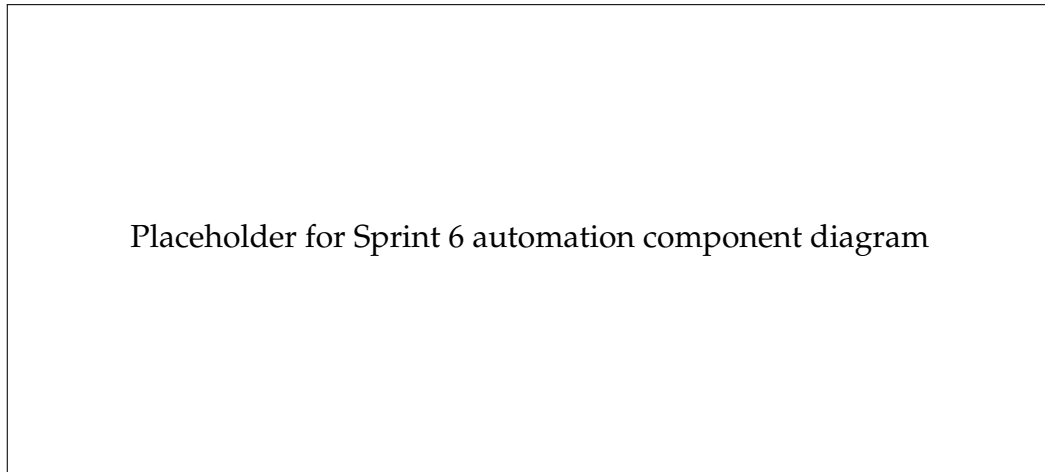


Figure 8.1: Components involved in backend-to-n8n-to-Telegram automation flow.

The automation path has five pieces: queue runtime metrics, webhook payload serializer, HTTP delivery client, n8n workflow router, and notification endpoints. Alert archival is routed back to the backend through a dedicated API endpoint.

8.3.2 Data Flow Diagram

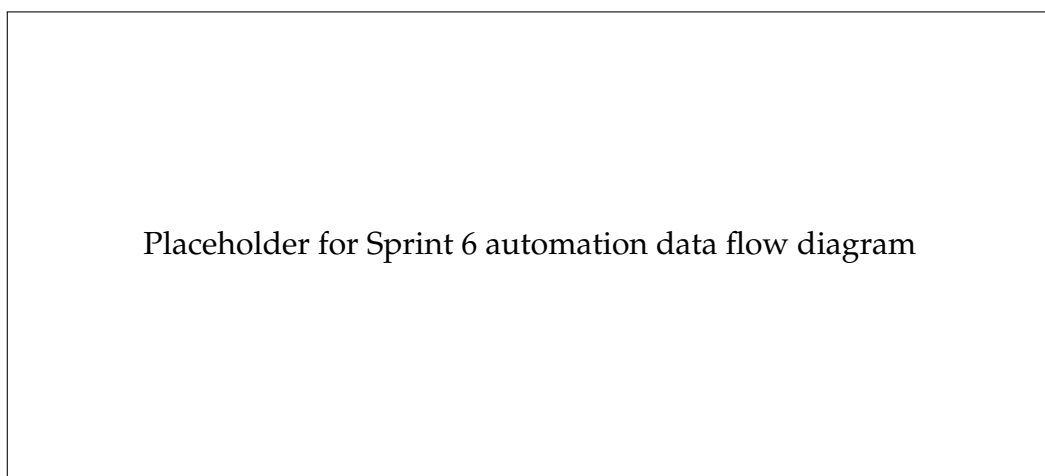


Figure 8.2: Alert payload flow from backend metrics to external notifications.

Data flow starts with periodic metrics dispatch from the runtime loop, branches on alert presence and cooldown in n8n, and then either sends Telegram notifications or returns acknowledgment without external dispatch. Archived alert payloads are persisted through `/api/alerts/archive`.

8.4 Implementation

8.4.1 Webhook Payload Contract

The webhook schema module defines `QueuePayload`, a structured contract carrying:

- source metadata (timestamp, frame, source, feed ID),
- queue metrics (λ , μ , wait time, stability),
- alert flags and severity fields.

The module includes payload validation before dispatch.

8.4.2 Reliable Webhook Delivery

The webhook delivery client sends payloads to n8n with timeout, retry policy, optional shared-secret header, and success/failure statistics. Payload persistence is handled through the SQLite database layer, keeping one source of truth for historical records.

8.4.3 Runtime Alert Dispatch Integration

In the main runtime loop, webhook sending is offloaded to `AsyncWebhookDispatcher` to avoid blocking the perception pipeline. The implementation supports periodic metric dispatch and immediate warning dispatch with deduplication windows to limit repeated alert spam.

8.4.4 n8n Workflow Routing and Telegram Delivery

The n8n alert workflow implements:

- webhook reception and secret validation,
- alert detection and cooldown gating,
- alert history archival via backend API,
- message formatting and Telegram send node,
- explicit acknowledge/reject branches.

That flow keeps the no-alert, alert-suppressed, and alert-delivered branches distinct enough to audit later.

8.4.5 Automation Testing Artifacts

Workflow validation is supported by dedicated PowerShell automation scripts for single-alert checks, automated execution polling, and multi-feed branch scenarios. These scripts automate webhook submission and branch-validation workflows during sprint testing.

8.5 Conclusion

Sprint 6 turned analytics output into actionable notifications and kept the historical trail inside the backend.

Sprint 7: Evaluation and Optimization

Table of Content

9.1	Introduction	58
9.2	Sprint Backlog	58
9.3	Design	58
9.3.1	Test Plan	58
9.3.2	Performance Metrics	59
9.4	Implementation	59
9.4.1	Automated Test Coverage	59
9.4.2	Runtime Optimization Actions	59
9.4.3	Resilience Improvements	60
9.4.4	Observed Outcome	60
9.5	Conclusion	60

9.1 Introduction

Sprint 7 acted as the validation pass for the full stack. The work checked whether perception, API, frontend, and alerting held together under tests, retries, and throughput pressure.

9.2 Sprint Backlog

Sprint 7 backlog included:

- consolidate backend and frontend automated test coverage,
- validate feed lifecycle, stream continuity, and API contracts,
- optimize runtime throughput and responsiveness,
- harden failure handling (network drops, stale data, worker exits),
- document residual limitations for final delivery planning.

9.3 Design

9.3.1 Test Plan

Validation followed a layered strategy:

- **Backend unit/integration tests:** queue analytics logic, API endpoints (including transport capability and WebRTC signaling), runtime streaming behavior, webhook payload handling.
- **Frontend integration tests:** dashboard setup flow, websocket event handling, WebRTC lifecycle/fallback behavior, and API error handling.

- **Scenario tests:** n8n send/acknowledge flows, unauthorized branch, and multi-feed alert scenarios.

9.3.2 Performance Metrics

The optimization targets were tied directly to operational requirements:

- processing responsiveness (loop cadence and processed-frame throughput),
- stream continuity under feed state transitions,
- webhook impact on main-loop latency,
- dashboard freshness under websocket degradation.

9.4 Implementation

9.4.1 Automated Test Coverage

The test suite covers API transport-capability and WebRTC-offer cases, runtime stream behavior, webhook delivery handling, dashboard integration, live-state synchronization, API error contracts, and transport fallback behavior. These tests verify API envelopes, event contracts, WebRTC signaling and fallback paths, stream handoff, and setup-workflow behavior.

9.4.2 Runtime Optimization Actions

Key optimizations implemented across the runtime loop and configuration layers include:

- configurable detector image size and frame stride,
- explicit inference device control and diagnostics,
- asynchronous webhook dispatch to keep the processing loop responsive,

- real-time playback pacing for file sources.

9.4.3 Resilience Improvements

Runtime resilience was improved at multiple levels:

- feed worker supervision and exit recovery in `FeedRegistry`,
- capture-to-worker frame handoff continuity in `FeedFrameStreamManager`,
- websocket reconnect/fallback logic on the frontend side.

9.4.4 Observed Outcome

By the end of the sprint, the system had reached a stable integration point where perception, analytics, API runtime, dashboard, and n8n automation operated as one pipeline. The remaining work is mostly benchmarking and deployment hardening rather than missing core functionality.

9.5 Conclusion

Sprint 7 closed the loop with structured testing and targeted performance improvements. The resulting platform is solid enough for demonstration and academic evaluation, with a clear path to production hardening.

Table of Content

10.1 Summary	62
10.2 Objective Review	62
10.3 Future Work	63

10.1 Summary

This end-of-studies project delivered a complete queue wait-time estimation platform that connects computer vision, queue analytics, web supervision, and automation workflows.

From a technical standpoint, the system now includes:

- a perception pipeline based on YOLO detection and ByteTrack identity continuity,
- queue-zone filtering and event-driven estimation of λ , μ , and expected wait time,
- stability-aware outputs with explicit queue-state semantics,
- a typed FastAPI backend-for-frontend with feed lifecycle control and live web-socket events,
- a React dashboard for monitoring, setup, and runtime interaction,
- n8n and Telegram integration for operational alert routing.

The resulting architecture demonstrates that a full monitoring-to-action loop can be achieved with modular components while keeping the system maintainable and testable.

10.2 Objective Review

The initial objectives are considered achieved at project scope level:

1. **Real-time detection and tracking:** implemented through dedicated detector and tracker modules.
2. **Queue-zone counting:** implemented with polygon-zone management and in-zone identity filtering.

3. **Queue metric estimation:** implemented for arrival rate, service rate, and wait time.
4. **Queue stability handling:** implemented through stable/unstable classification and fallback wait-time logic.
5. **Web monitoring:** implemented through a multi-feed dashboard with live updates.
6. **Automation integration:** implemented through webhook transport, n8n routing, and Telegram notifications.

The main remaining gap is not architectural completeness, but depth of long-horizon evaluation under broader operational scenarios.

10.3 Future Work

Potential extensions include:

- multi-zone and multi-server queue modeling (for parallel cashier configurations),
- richer alert strategies (cooldown tuning, escalation policies, acknowledgment loops),
- edge deployment optimization (ONNX/TensorRT profiling and resource-aware scheduling),
- long-duration field evaluation with domain-specific KPI baselines,
- stronger security controls for production webhook exposure and secrets management.



References

- [1] D. Gross, J. F. Shortle, J. M. Thompson, and C. M. Harris, *Fundamentals of Queueing Theory*, 5th ed. Wiley, 2018.
- [2] Ultralytics, *Ultralytics documentation*, <https://docs.ultralytics.com/>, 2026.
- [3] Y. Zhang et al., “Bytetrack: Multi-object tracking by associating every detection box,” *arXiv preprint arXiv:2110.06864*, 2022.
- [4] OpenCV, *Opencv documentation*, <https://docs.opencv.org/>, 2026.
- [5] W3C Web Real-Time Communications Working Group, *WebRTC 1.0: Real-time communication between browsers*, <https://www.w3.org/TR/webrtc/>, Accessed: 2026-04-07, 2025.
- [6] MediaMTX, *Mediamtx documentation*, <https://mediamtx.org/docs/>, Accessed: 2026-04-07, 2026.
- [7] Internet Engineering Task Force, *Rfc 8830: WebRTC media transport and use of rtp*, <https://www.rfc-editor.org/rfc/rfc8830>, Accessed: 2026-04-07, 2021.



A.1 Useful Commands

- Run backend API: `uvicorn backend.app:app -reload`
- Run backend tests: `pytest tests -q`
- Run frontend dev server: `cd frontend && npm run dev`
- Run frontend tests: `cd frontend && npm run test`

A.2 Example Webhook Payload

```
{
  "timestamp": "2026-03-05T14:32:10.123Z",
  "camera_id": "cam_01",
  "zone_id": "checkout_lane_3",
  "metrics": {
    "people_in_zone": 7,
    "arrival_rate": 0.15,
    "service_rate": 0.16,
    "wait_time_seconds": 15.0,
    "queue_stable": true
  },
  "alerts": [
    {
```

```
    "type": "WAIT_TIME_WARNING",
    "severity": "warning",
    "message": "Wait time exceeded 60s threshold",
    "value": 72.3,
    "threshold": 60
  }
],
"raw_detection_count": 7,
"fps": 24.5
}
```

A.3 Items to Add Before Final Submission

- final versions of all architecture and sequence diagrams,
- finalized evaluation tables and plots for Sprint 7,
- appendix references for API and test artifacts,
- proofreading pass for terminology and citation consistency.